

SLOVENSKÁ TECHNICKÁ UNIVERZITA V BRATISLAVE

Fakulta elektrotechniky a informatiky

Zadanie č.2
Kalibrácia Kamery

Počítačové videnie a spracovanie obrazu

Obsah

1	Zadanie	3
1.1	Znenie zadania	3
1.2	Úlohy.....	3
1.2.1	Kalibrácia kamery [2,5 b.]	3
1.2.2	Detekcia geometrických tvarov [2,5 b.].....	3
1.2.3	Farebný filter [1,5 b.]	4
1.2.4	Dokumentácia [1,5 b.].....	4
2	Riešenie	5
2.1	Kalibrácia kamery	5
2.2	Detekcia geometrických tvarov	7
2.3	Farebný filter.....	9
	Záver	13

1 Zadanie

1.1 Znenie zadania

V tomto zadaní budete pracovať s 2D kamerou od spoločnosti Ximea. Cieľom je:

- realizovať kalibráciu kamery,
- aplikovať získané parametre na reálny obraz,
- implementovať detekciu geometrických tvarov,
- precvičiť si prácu s dokumentáciou OpenCV

1.2 Úlohy

1.2.1 Kalibrácia kamery [2,5 b.]

Realizujte kalibráciu kamery pomocou šachovnice (chessboard pattern) podľa oficiálneho postupu v OpenCV.

Po kalibrácii:

- vypíšte maticu vnútorných parametrov kamery,
- určte hodnoty f_x , f_y , c_x , c_y ,
- uložte maticu kamery a distorzné koeficienty pre ďalšie použitie,
- demonštrujte odstránenie skreslenia (undistortion) na reálnom obraze.

Výsledkom musí byť funkčný program, ktorý vykoná kalibráciu z viacerých snímok šachovnice a vypočíta vnútorné parametre kamery.

1.2.2 Detekcia geometrických tvarov [2,5 b.]

Vytvorte program na detekciu základných geometrických tvarov (napr. kružnica, štvorec, trojuholník, obdĺžnik) v obraze pomocou vhodných metód dostupných v knižnici OpenCV (napr. Houghova transformácia, detekcia hrán, kontúry a ich aproximácia).

Program musí:

- detegovať tvary v reálnom obraze z kamery (Pomenovať),
 - graficky označiť detegovaný tvar,
- zobrazit' stred (resp. ťažisko) tvaru,

Pri odovzdávaní demonštrujte detekciu na rôznych typoch tvarov (rôzna veľkosť, farba, osvetlenie, poloha v obraze).

1.2.3 Farebný filter [1,5 b.]

Implementujte farebný filter, ktorý:

- vyberie jednu konkrétnu farbu v obraze (napr. červenú),
- všetky pixely tejto farby nahradí inou zvolenou farbou.

Príklad: všetky červené objekty budú zobrazené ako zelené. Farbu detegujte vhodným spôsobom (prahovanie vo vhodnom farebnom priestore). Výsledok musí fungovať v reálnom čase na obraze z kamery

1.2.4 Dokumentácia [1,5 b.]

Vypracujte stručnú dokumentáciu vo formáte PDF, ktorá bude obsahovať:

- postup riešenia,
- použité metódy,
- ukážky výsledkov (obrázky),
- získané hodnoty parametrov kamery

2 Riešenie

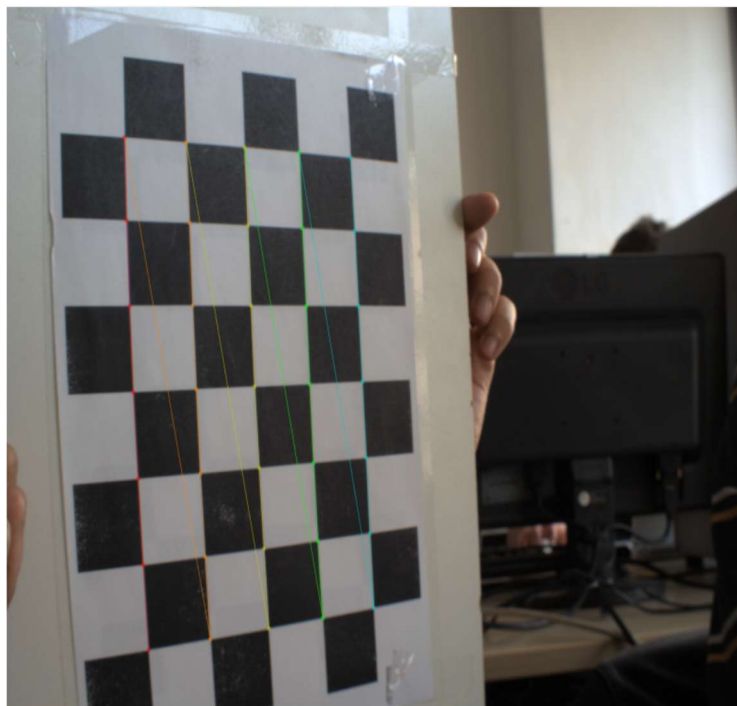
2.1 Kalibrácia kamery

Kalibrácia kamery slúži na to aby sme presne vedeli presne popísať ako objektív skresľuje obraz. Získavame maticu vnútorných parametrov f_x , f_y , c_x , c_y a vektor skreslenia. Tie neskôr používame pri použití undistortion neskôr. Postup je založený na šachovnici, pretože má veľa jednoznačných rohov. Najprv sa vytvorí dataset snímok skript s Ximea kamerou zobrazí live obraz a po stlačení klávesy uloží snímku do priečinka dataset5. Dôležité je mať viac snímok šachovnice v rôznych polohách, natočeniach a aj pri okrajoch obrazu, aby sa dalo dobre odhadnúť skreslenie objektívu.

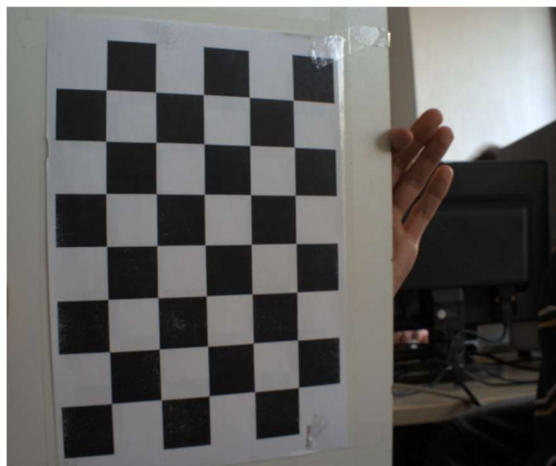
V kalibračnom skripte potom sa pre každú snímku z datasetu spraví prevod do čierneho-bielej a funkciou `findChessboardCorners` sa nájdu 2D súradnice rohov v pixeloch. Zároveň sa vytvoria 3D body šachovnice. Pre každú úspešnú snímku sa uloží dvojica: `objpoints` (3D body) a `imgpoints` (2D body v obraze).

Funkcia `calibrateCamera` následne odhadne parametre kamery tak aby pri projekcii 3D bodov do obrazu vychádzali čo najbližšie k reálne detegovaným rohom.

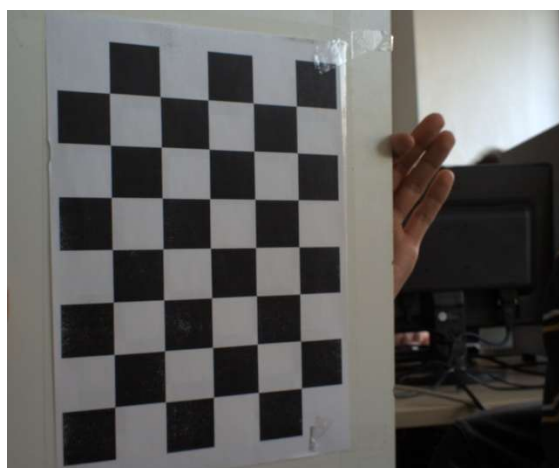
Kalibračný skript je v súbore `Calibration.py` a je len rozšíreným kódom z návodu OpenCV.



Obr. 1: Vyznačené rohy šachovnice 7x5



Obr. 2: Pôvodný obrázok



Obr. 3: Skalibrovaný obrázok

3807.90520	0	1234.89791
0	3800.83856	1043.67850
0	0	0

Tabuľka vnútorných parametrov kamery

-4.05883226e-01	3.56003922e-01	1.31229760e-04	8.01290174e-04	1.01090002
-----------------	----------------	----------------	----------------	------------

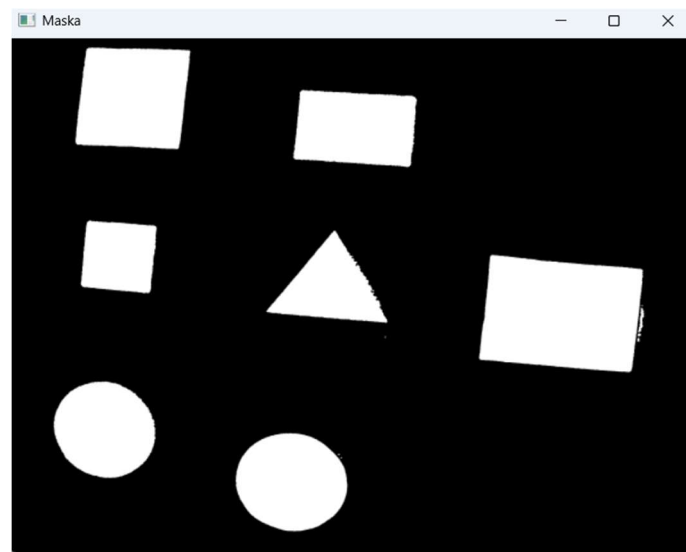
Distorzné koeficienty

2.2 Detekcia geometrických tvarov

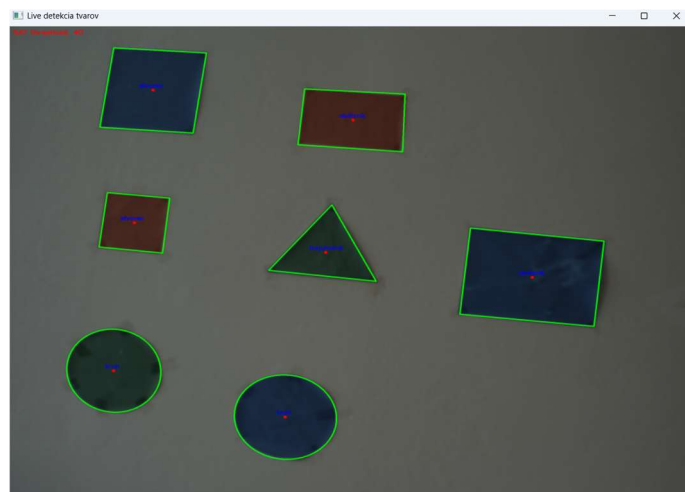
V druhej časti sme vytvorili program, ktorý slúži na detekciu základných geometrických útvarov (napr. štvorec, kruh, obdĺžnik a trojuholník). V počiatočnej fáze prevedieme obraz z farebného priestoru BGR na HSV, aby sme mohli využiť kanál saturácie pre aplikovanie Gaussovho rozmazania a prahovania na vytvorenie binárnej masky. Táto maska slúži na zvýraznenie farebných objektov na pozadí. Na túto masku sa aplikujú aj morfologické operácie, pomocou ktorých odstraňujeme drobný šum a veci podobné aby sme získali čistejšiu masku. Z pripravenej masky sa následne vyhľadajú kontúry jednotlivých objektov. Na určenie typu tvaru využívame aproximáciu obrysov aby sme lepšie zistili počet vrcholov tvaru.

- 3 vrcholy – trojuholník
- 4 vrcholy – štvorec alebo obdĺžnik
- Viac ako 4 – kruh

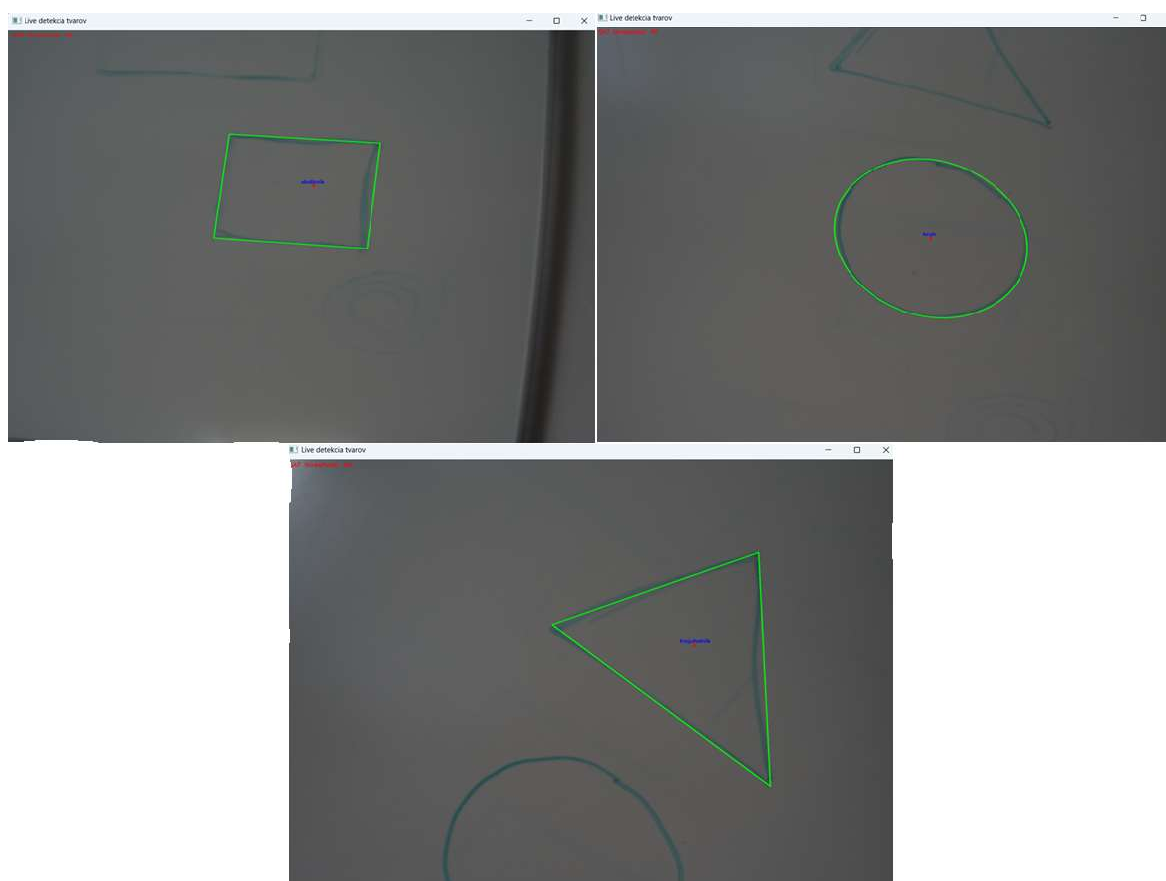
Pri detekcii kruhu taktiež využívame aproximáciu elipsou, oblosť a pomer strán, aby bola detekcia stabilnejšia. Následne sa do obrázku vykreslí obrys detegovaného tvaru, ťažisko a názov.



Obr. 4: Vytvorená binárna maska



Obr. 5: Live nasnímané objekty v triede



Obr. 6: Live nasnímané objekty z tabule

Použité cv2 metódy:

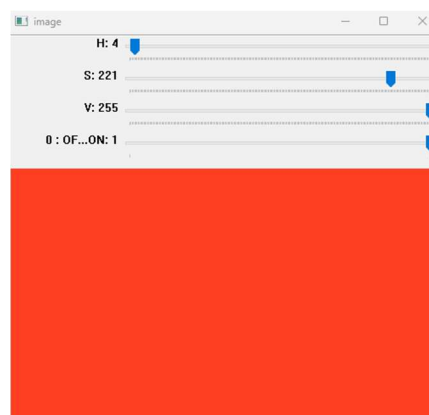
- cv2.boundingRect()
- cv2.moments()

- cv2.cvtColor()
- cv2.threshold()
- cv2.getStructuringElement()
- cv2.morphologyEx()
- cv2.contourArea()
- cv2.convexHull()
- cv2.approxPolyDP()
- cv2.fitEllipse()
- cv2.findContours()
- cv2.findContours()

2.3 Farebný filter

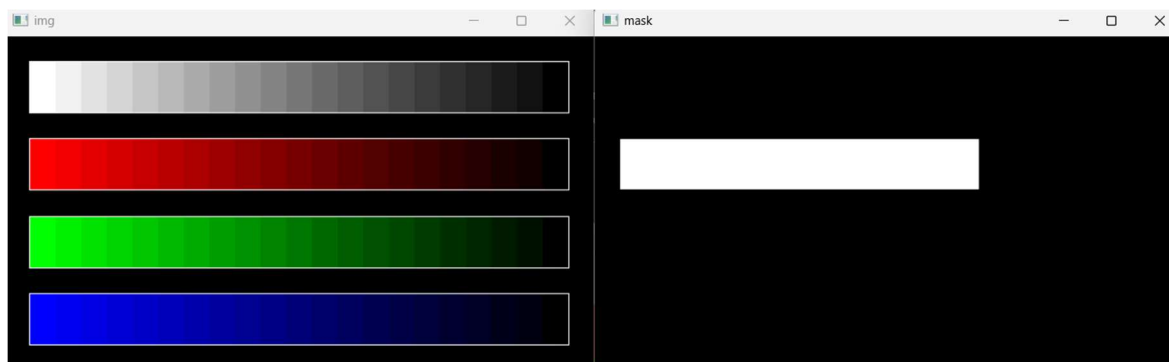
Farebný filter má za úlohu upraviť nejakú farbu v určitom spektre na inú požadovanú farbu. Primárne musí program najprv určiť v obraze, ktoré pixeli sa nachádzajú v požadovanom spektre. Na základe toho sa potom vytvorí maska obrazu, prostredníctvom, ktorej je možné potom v pôvodnom obraze farbu prepísať na inú.

Pri tejto úlohe sme sa rozhodli filtrovať červenú farbu. Pri zobrazení farieb v RGB/BGR formáte je však náročné určiť koľko ktorej zložky je potrebné aby to bola naša požadovaná farba, preto je nutné previesť obrázok do HSV formátu. Pre jednoduchosť hľadania vhodného rozsahu sme upravili kód z OpenCV na TrackBar pre vykreslenie BRG zložiek na HSV formát.

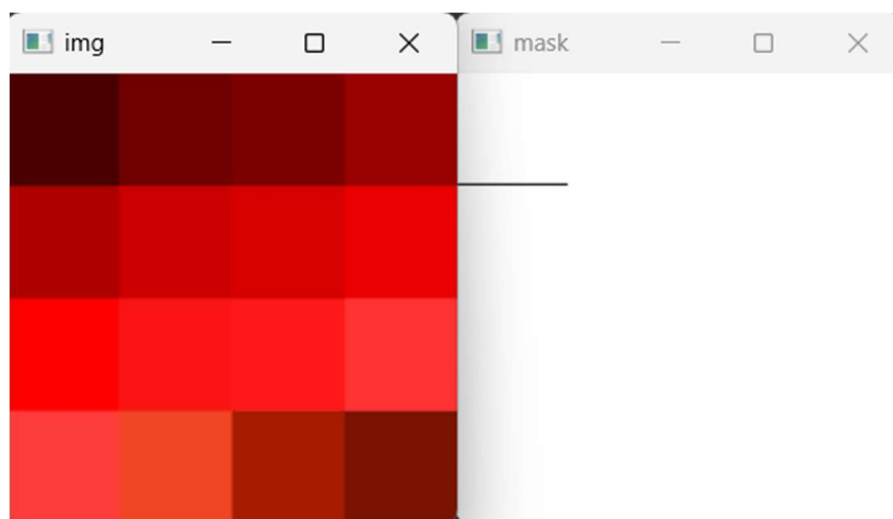


Obr. 7: Okno upravené trackBar kódu z OpenCV

Masku sme nastavili ako dostatočný rozsah na červenú $H = [0-10]$, $S = [150 - 255]$, $V = [75 - 255]$. Pri detekcii červenej nám vznikali nasledovné masky.

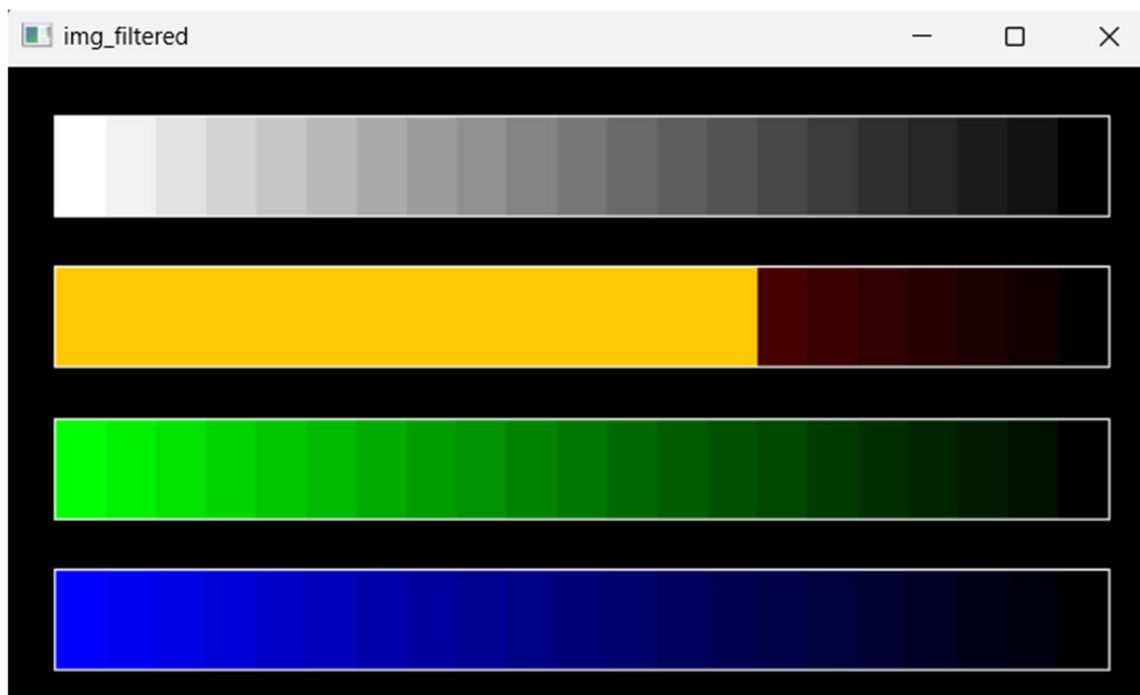


Obr. 8: Porovnanie pôvodného obrázku a nami zvolenej masky



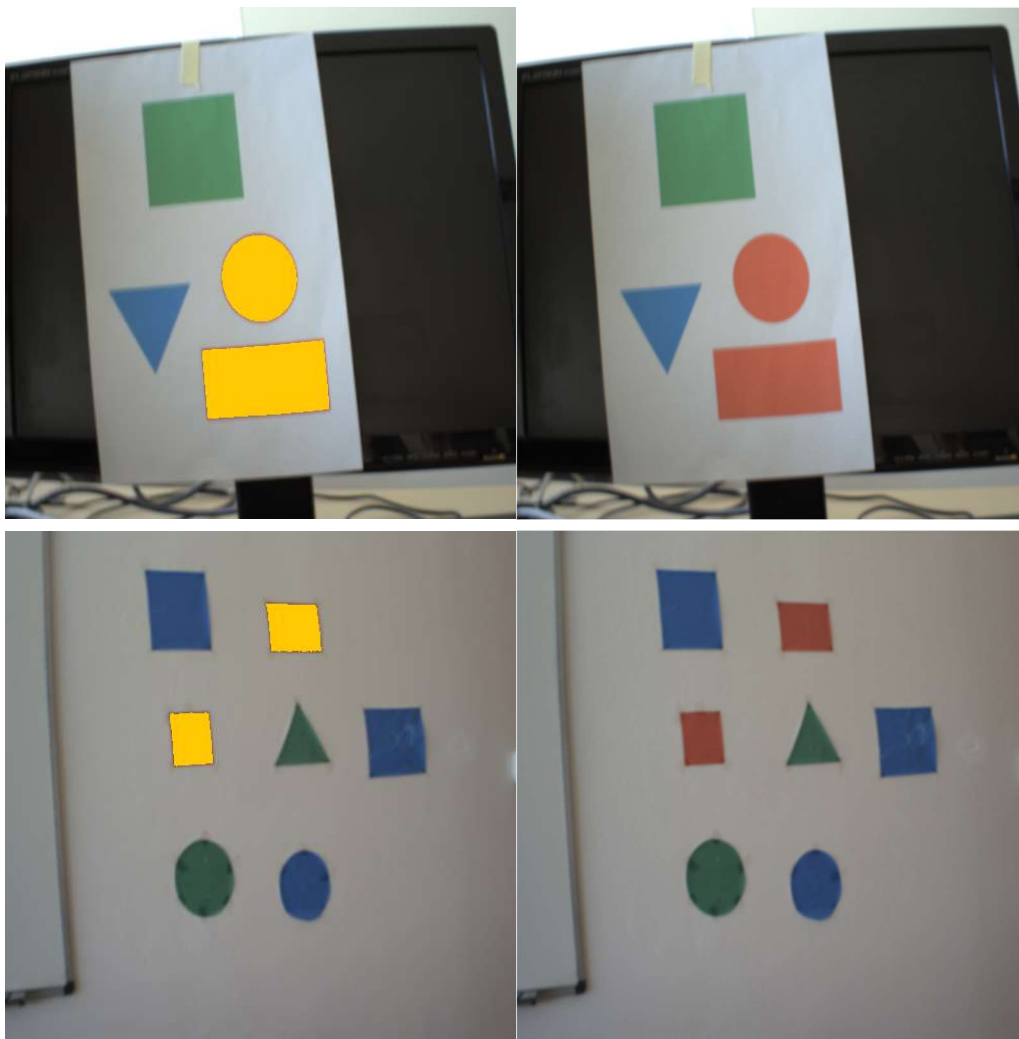
Obr. 9: Porovnanie pôvodného obrázku a nami zvolenej masky

Na základe tejto masky už stačí ak do hodnoty v príslušných poliach pôvodného obrázku prepíšeme na žltú (0, 200, 255).



Obr. 10: Odfiltrovanie červenej farby na žltú

Ak máme takto spracovaný statický obraz stačí, tak rovnako upravíme každú snímku z kamery čím vznikne živý obraz z kamery s farebným filtrom.



Obr. 11: Odfiltrovanie červenej farby na žltú na živom obraze z kamery

Záver

V rámci tohto zadania, na ktorom sme pracovali s 2D kamerou od spoločnosti Ximea, sa nám podarilo úspešne zvládnuť všetky tri časti zadania. Pri kalibrácii boli získané vnútorné parametre kamery a distorzné koeficienty, ktoré umožnili odstránenie skreslenia obrazu. Pri detekcii tvarov sme úspešne vytvorili binárnu masku, kde sme následne využili rôzne metódy na určenie jednotlivých tvarov a následne sme využili tieto metódy aj na Live snímaní. Pri poslednej úlohe sa nám úspešne podarilo, vytvoriť identifikáciu farebného rozsahu na statickom obrázku a na základe masky konkrétne pixely upraviť. V tomto zadaní sme si precvičili praktické využitie vecí z knižnice OpenCV pri spracovaní obrazu a práci s kamero-
vým systémom.